

Epistemic Computing: Separating Algebraic Certainty from Measurement Uncertainty in Scientific Programming

Demetrios C. Agourakis

Biomaterials and Regenerative Medicine Post-Graduate Program,
Pontifícia Universidade Católica de São Paulo (PUC-SP), Sorocaba, SP, Brazil
ORCID: 0009-0001-8671-8878

March 2026

Correspondence: demetrios@agourakis.med.br

Abstract. Scientific computation routinely combines exact mathematical structure (symmetry classifications, conservation laws, algebraic identities) with empirical measurements subject to uncertainty. In current practice, both are represented as floating-point numbers, making the boundary between “certain because proven” and “uncertain because measured” invisible to the program and often to the programmer. We argue that this conflation requires a programming language with *epistemic types*: types that carry uncertainty metadata through every computation, following the GUM (Guide to the Expression of Uncertainty in Measurement) standard. We describe how the Sounio programming language — a systems language with native `Knowledge<T>` types, compile-time dimensional analysis, an algebraic effect system, and refinement types — provides such a framework. Through worked examples in crystallography (exact space group symmetry vs. uncertain diffraction intensities), we show how exact and uncertain quantities coexist in the same type system, with the compiler enforcing that uncertainty is never silently discarded. This approach applies broadly to any domain where exact mathematical structure meets empirical measurement uncertainty — including pharmacokinetics, genomics, and climate modelling.

Keywords: epistemic computing, uncertainty quantification, type systems, GUM, scientific programming, crystallography

MSC/ACM: 68N18, 92C45, 62P10

1. The problem: two kinds of knowledge in one computation

Consider a crystallographer determining a molecular structure from X-ray diffraction data. Two fundamentally different kinds of knowledge are involved:

Algebraic knowledge (exact): “This crystal belongs to space group $Fm\bar{3}m$ (No. 225). This is determined by symmetry and is not subject to measurement error.”

Empirical knowledge (uncertain): “The measured diffraction intensity at reflection $(2, 1, 3)$ is 1247 ± 35 counts/s. The derived bond length is 1.542 ± 0.003 Å.”

In current practice, both kinds of knowledge are represented as floating-point numbers in Python, R, or Fortran. The space group number 225 and the bond length 1.542 have the same type (`int` or `float`). The uncertainty ± 0.003 is tracked in comments, CIF metadata, or not at all.

This conflation has consequences. When exact symmetry constraints propagate into uncertain structure refinement, the boundary between “certain because proven” and “uncertain because measured” is invisible to the program, the compiler, and often the programmer. Downstream computations silently combine exact and uncertain quantities, producing results whose confidence is unknown.

2. Background

2.1. The 168 theorem and CYP450 mapping

In companion papers [1–3], we established that:

- The number of nonzero basis associator triples in the Cayley–Dickson tower is always a multiple of $168 = |\text{PSL}(2, 7)|$.
- The 7 FDA-canonical CYP450 isoforms map to the Fano plane, classifying 343 enzyme triples into 133 trivial, 42 associative, and 168 non-associative.
- The prediction is testable: DDI order-dependence should correlate with the Fano classification.

The algebraic layer is exact and verified exhaustively through dimension 64. The clinical layer is where uncertainty enters.

2.2. The GUM framework

The *Guide to the Expression of Uncertainty in Measurement* (GUM) [4] provides the international standard for propagating measurement uncertainty through calculations. For a function $f(x_1, \dots, x_n)$ of measured quantities x_i with standard uncertainties $u(x_i)$, the combined standard uncertainty is:

$$u(f) = \sqrt{\sum_{i=1}^n \left(\frac{\partial f}{\partial x_i} \right)^2 u(x_i)^2} \quad (1)$$

GUM uncertainty propagation is mathematically well-defined but rarely enforced by programming tools. Most implementations require manual tracking of uncertainty through separate variables.

3. Epistemic types in Sounio

The Sounio programming language [5] introduces `Knowledge<T>` as a first-class type. A value of type `Knowledge<T>` carries:

- A *central value* of type `T`
- A *standard uncertainty* u (following GUM)
- An optional *provenance* tag identifying the uncertainty source

Arithmetic on `Knowledge<T>` values automatically propagates uncertainty via Equation 1. The compiler enforces that:

1. `Knowledge<T>` values cannot be silently cast to `T` (uncertainty cannot be discarded).
2. Exact values (type `T`) can be promoted to `Knowledge<T>` with $u = 0$.
3. Dimensional analysis is enforced at compile time via the unit system.

3.1. Example: algebraic certainty

```
// Crystal space group determination is exact.
// identify_space_group returns i32 (one of 230 groups).
// There is no uncertainty – it's a mathematical classification.
```

```
let group: i32 = identify_space_group(unit_cell)
// = 225 (Fm-3m, face-centered cubic). Exact. No ±.
```

The return type `i32` carries zero uncertainty by construction. The compiler knows this value is exact — it follows from symmetry, not from measurement.

3.2. Example: measurement uncertainty

```
// X-ray diffraction intensity has measurement uncertainty
// from counting statistics, background subtraction, absorption.
```

```

let intensity: Knowledge<counts_per_sec>
  = measure(1247.0, uncertainty: 35.0)

// Structure factor depends on intensity.
// Uncertainty propagates automatically (GUM).

let F_obs: Knowledge<f64>
  = structure_factor(intensity, lorentz_correction, scale)
// F_obs.value = 35.3, F_obs.uncertainty = 0.5

// Electron density from Fourier synthesis.
let rho: Knowledge<e_per_A3> = fourier_sum(F_obs, phases)
// rho.value = 2.14, rho.uncertainty = 0.08

```

Every intermediate value carries its uncertainty. The compiler rejects any attempt to use `rho` as a plain `f64` — the uncertainty must be explicitly acknowledged.

3.3. Example: the bridge — exact structure meets uncertain measurement

```

// THE KEY COMPUTATION: exact symmetry + uncertain measurement.

let space_group: i32 = identify_space_group(unit_cell)
// Exact. Algebraic. No uncertainty.

let bond_length: Knowledge<angstrom> = if has_inversion_center(space_group) {
  // Centrosymmetric: phases are 0 or  $\pi$  (exact).
  // Bond length uncertainty comes only from intensity.
  refine Centrosymmetric(F_obs, phases)
} else {
  // Non-centrosymmetric: phases are uncertain.
  // Bond length carries ADDITIONAL uncertainty from phase ambiguity.
  refine Noncentrosymmetric(F_obs, estimated_phases)
}

// The compiler knows:
//   space_group has type i32 (exact, algebraic)
//   bond_length has type Knowledge<angstrom> (uncertain, measured)
// The exact classification gates the computation.
// The uncertainty propagates through the branch.

```

The algebraic classification (space group symmetry) gates the computational method, but the types remain distinct. The `if` branch is determined by exact mathematics; the result carries measurement uncertainty. A downstream function receiving `bond_length` knows, from its type alone, that this value is uncertain and must be treated accordingly.

4. Additional type-system features

4.1. Compile-time units

Sounio’s unit system prevents dimensional errors at compile time:

```

let dose: mg = 500.0
let volume: L = 0.35
let concentration: mg_per_L = dose / volume // type-checked
// let error = dose + volume                // COMPILE ERROR: mg + L

```

For CYP450 pharmacokinetics, this means clearance rates (`mL/min`), AUC values (`ng·h/mL`), and doses (`mg`) cannot be accidentally combined without dimensional consistency.

4.2. Effects

Sounio’s algebraic effect system tracks computational side effects:

```

fn classify_triple(a: i32, b: i32, c: i32) -> i32 with Mut, Panic {
  // Pure computation. No IO, no network, no randomness.
}

```

```

    // The 'with' clause proves this function's result depends
    // only on its inputs – it is algebraically deterministic.
}

fn predict_order_dependence(g: Genotype, t: Timing)
  -> Knowledge<f64> with Mut, Panic, IO {
  // This function may read patient data (IO effect).
  // The compiler enforces that callers handle the IO effect.
}

```

The effect system makes the boundary between pure mathematics and empirical data visible at the type level.

4.3. Refinement types

```

type BasisIndex = { x: i32 | x >= 1 && x <= 7 }
type FanoLine = { t: [BasisIndex; 3] | is_on_fano_line(t[0], t[1], t[2]) }

```

The compiler rejects any attempt to construct a `FanoLine` from elements that don't form a valid Fano line. The algebraic structure is enforced at compile time, not by runtime checks.

4.4. Linear types for physical resources

```

linear struct Sample { id: SampleId, remaining: Knowledge<mg> }

fn mount_and_expose(sample: Sample, beam: &XrayBeam) -> Dataset with IO {
  // 'sample' is consumed by X-ray exposure.
  // The linear type ensures:
  //   - The sample cannot be exposed twice (radiation damage)
  //   - The sample cannot be silently dropped (data loss)
}

```

A crystal sample is a *linear resource*: it is consumed by X-ray exposure. The compiler enforces single-use at the type level. The same pattern applies to reagent aliquots, patient samples, and drug doses — any physical resource that must be used exactly once.

5. The epistemic pipeline: from algebra to clinic

Putting these features together, the full pipeline from mathematical theorem to clinical prediction is:

Layer	Type	Uncertainty
Space group classification	i32	None (exact)
Diffraction intensity	Knowledge<counts/s>	GUM from counting statistics
Absorption correction	Knowledge<f64>	From crystal shape measurement
Structure factor F_{obs}	Knowledge<f64>	Propagated from intensity + corrections
Electron density ρ	Knowledge<e/Å ³ >	Propagated from F + phases
Bond length	Knowledge<Å>	Propagated from all above

Table 1: The epistemic pipeline. Each layer has a type that encodes its uncertainty status. The compiler enforces that no layer silently discards uncertainty from upstream.

The first layer (Fano classification) is exact. Every subsequent layer introduces or propagates uncertainty. The final clinical prediction carries the combined uncertainty from every measurement and model parameter, following GUM. At no point can the programmer accidentally treat an uncertain quantity as exact — the type system prevents it.

6. Discussion

The approach described here is not specific to CYP450 pharmacology. Any domain where exact mathematical structure meets empirical measurement uncertainty would benefit from epistemic types. Examples include:

- **Crystallography**: space group classification (exact) combined with diffraction intensity measurement (uncertain).
- **Genomics**: sequence alignment scoring (algorithmically exact) combined with base-calling quality (uncertain).
- **Climate modelling**: conservation laws (exact) combined with parameterised sub-grid processes (uncertain).

The key insight is that uncertainty is not a property of numbers — it is a property of *how we know* those numbers. A programming language that tracks *how we know* alongside *what we know* prevents a class of errors that no amount of runtime validation can catch: the silent conflation of certainty with ignorance.

Crystallography provides an unusually clean example: the space group classification is provably exact (it follows from symmetry), while the derived structural parameters (bond lengths, angles, electron densities) depend on measured intensities with irreducible uncertainty. Sounio’s type system holds both, without confusing them. The same separation applies to any scientific domain where mathematical structure meets empirical measurement.

7. Availability

The Sounio programming language, including the `Knowledge<T>` type and the unit system, is available at github.com/agourakis82/sounio.

Bibliography

1. Agourakis, D. C., Gerenutti, M.: The 168 Count for Octonion Basis Associators and a Computational Cayley–Dickson Tower Extension. *Advances in Applied Clifford Algebras*. (2026)
2. Agourakis, D. C.: The 168 Quantum of Non-Associativity in the Cayley–Dickson Tower, (2026)
3. Agourakis, D. C.: $PG(k-1, 2)$ Geometry as a Mathematical Framework for Order-Dependent Processes in Biology, (2026)
4. JCGM: Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement. Joint Committee for Guides in Metrology (2008)
5. Agourakis, D. C.: Sounio: A systems programming language for epistemic computing, <https://github.com/agourakis82/sounio>